

Tunable Consistency and Client-Centric Guarantees in MongoDB

Project 1 · Complete Experimental Report

Prepared by: Pan Yuxiang, Hou Jiacheng, and Wu Jiarui
Experimental platform: Google Cloud Compute Engine (Singapore)

Main finding: causal sessions with majority reads and writes preserved all four guarantees for successful operations; without a causal session, even majority/majority returned successful stale reads under controlled replication lag.

Repository: github.com/Panyuxiang0830/DSA5208_project1

Table of Contents

Table of Contents	2
Abstract	3
1. Objectives and Research Questions	3
2. Background: Four Client-Centric Guarantees	3
3. Database and Deployment Architecture	4
3.1 Software and Cloud Resources	4
3.2 Installation and Initialization	4
4. Consistency Configurations and Predictions	5
5. Experimental Design	5
5.1 Workloads and Measurement	5
5.2 Controlled Comparisons and Reuse	6
5.3 Fault Injection and Recovery	6
6. Results	6
6.1 S0-S3: Stable-State Matrix	6
6.2 Non-Causal Read/Write Concern Matrix	7
6.3 Latency	8
6.4 T1/T2: Election Transition Windows	8
6.5 S4: Controlled Replication Lag	10
7. Predictions Versus Observations	10
7.1 Representative Configurations	10
7.2 Controlled Concern Matrix	11
8. Limitations and Threats to Validity	11
9. Reproducibility	12
9.1 Main Commands	12
9.2 Formal Run Identifiers	12
10. Conclusion	13
References	13
AI Usage Statement	13

Abstract

This project deploys a three-member MongoDB 8.0.29 replica set with Docker Compose on one Google Cloud virtual machine and studies how read/write concerns, read preference, and causal sessions affect four client-centric guarantees. The suite covers normal operation, node failures, a network partition, election windows, and controlled replication lag. Formal baseline and stable-fault runs use 500 sequences and three seeds per configuration-model pair. C1, C2, and C4 showed no violations in successful checks. Weak configuration C3 produced 76.07% read-your-writes violations in normal operation and 64.60%-69.93% under stable faults. Elections caused transient operation errors but no new successful-read anomaly in strong configurations. Under S4, C3 read-your-writes and monotonic-read violation rates rose to 83.33% and 49.80%; non-causal C8 still showed 58.87% read-your-writes and 49.80% monotonic-read violations despite majority/majority concerns. The approximately 50% writes-follow-reads (WFR) flags in S4 largely arise when a paused replica returns no document and the checker substitutes version -1. They are not a valid WFR violation rate: the experiment did not establish that a later write executed before a version actually read. The results distinguish session order from replica visibility, while zero observed violations are not a proof over all histories.

Keywords: MongoDB; replica sets; tunable consistency; causal consistency; fault injection; replication lag

1. Objectives and Research Questions

The assignment requires a replicated database with tunable consistency, predictions about client-observed guarantees, and experiments under normal operation, node failures, and a network partition. MongoDB was selected because its driver exposes sessions, read concern, write concern, and read preference, allowing abstract consistency models to be mapped to reproducible client configurations.

- Which configurations preserve read-your-writes (RYW), monotonic reads (MR), monotonic writes (MW), and writes-follow-reads (WFR)?
- With non-causal sessions and Secondary read order held fixed, how do read concern and write concern change the anomalies observed by the client?
- How do stable degraded states and election transition windows differ in their effects on consistency and availability?
- Can controlled replication lag amplify occasional stale reads into stable, explainable anomalies?

2. Background: Four Client-Centric Guarantees

The four guarantees are complementary session guarantees within causal consistency, not a simple one-dimensional strength ranking. A system may satisfy one while violating another; causal consistency is commonly characterized by all four together. The table gives the executable checks used here.

Model	Application expectation	Operational check
RYW	After writing, the client reads that version or newer.	<code>observed_version < acknowledged_version</code>
MR	Successive reads by one client never go backwards.	<code>later_version < earlier_version</code>
MW	Writes from one client take effect in issue order.	<code>acknowledged history/final version out of order</code>
WFR	A dependent write is ordered after the version it read.	<code>parent missing/stale predicate/dependent write missing</code>

3. Database and Deployment Architecture

3.1 Software and Cloud Resources

Item	Configuration
Cloud	Google Cloud Compute Engine; project nifty-pursuit-505911-n0
VM	dsa5208-mongodb; asia-southeast1-b (Singapore); e2-standard-2; 2 vCPU; 8 GB RAM
OS	Ubuntu 24.04.4 LTS; Linux 6.17.0-1022-gcp
Storage	30 GB balanced persistent disk; MongoDB named volumes
Containers	Docker Engine 29.7.2; Docker Compose v5.5.0
Database	MongoDB Community 8.0.29; three data-bearing voting members; no arbiter
Client	Python 3.12-slim; PyMongo 4.17.0
Limits	1536 MB RAM per mongod container; 0.5 GB WiredTiger cache

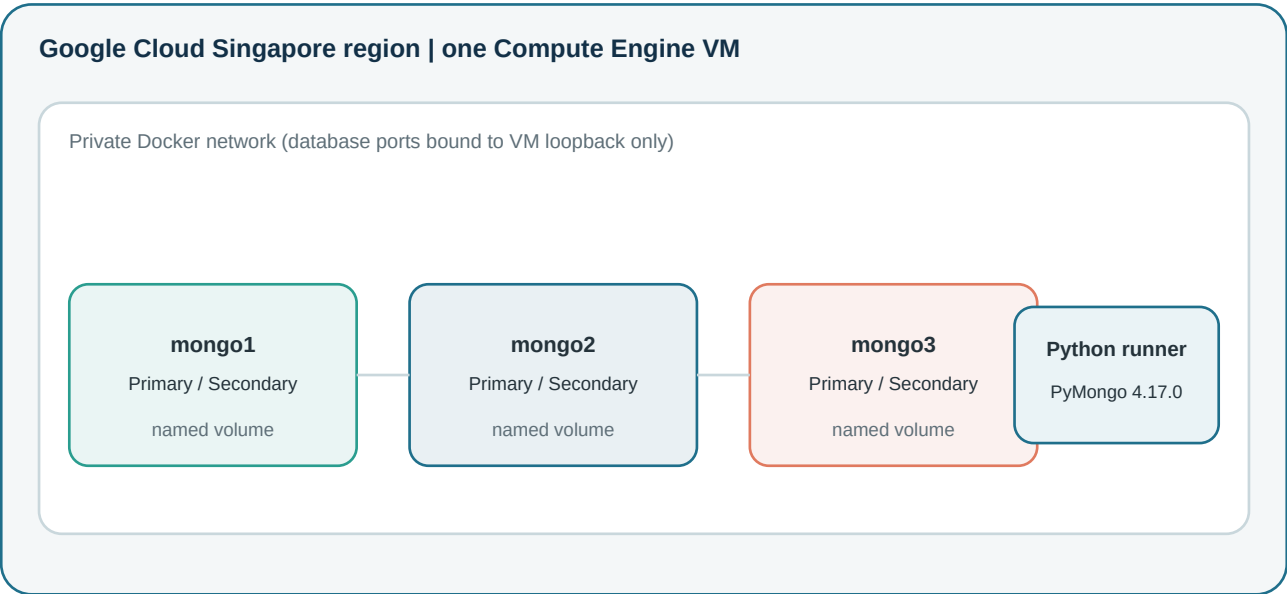


Figure 1. Experimental deployment. The three MongoDB members use separate containers, processes, and persistent volumes, but share one VM.

This design provides a logically replicated deployment and deterministic container-stop and container-network fault injection. Database ports are bound only to VM loopback and are not exposed directly to the public network. Its main trade-off is shared host, disk, and underlying network resources, so it does not model physical fault independence.

3.2 Installation and Initialization

```
git clone https://github.com/Panyuxiang0830/DSA5208_project1.git
cd DSA5208_project1
./scripts/bootstrap-ubuntu.sh
cp .env.example .env
docker compose pull
docker compose up -d --wait mongo1 mongo2 mongo3
docker compose run --rm mongo-init
./scripts/cluster_status.sh
docker compose build runner
docker compose run --rm --no-deps runner
```

The initialization script creates three voting, data-bearing replica-set members. Under normal operation one member is Primary and two are Secondary; MongoDB's replica-set protocol controls elections. Removing or restarting containers does not remove named volumes, so database files survive normal recovery procedures.

4. Consistency Configurations and Predictions

MongoDB write concern controls when replication acknowledgment is sufficient, read concern controls the visibility level that reads may observe, read preference routes reads to primary or secondary members, and causal sessions carry causal time through the driver. MongoDB documents that causally consistent sessions combined with majority read and write concerns provide RYW, MR, MW, and WFR across members.

Config	Session	Write concern	Read concern	Read preference	Prediction
C1	Causal	majority	majority	primary	All four hold
C2	Causal	majority	majority	secondary	All four hold; may wait for replica
C3	Non-causal	w: 1	local	secondaryPreferred / Secondary	RYW/MR/WFR may fail; MW usually holds
C4	Non-causal	majority	majority	primary	Expected to hold here; not equivalent to general causal session
C5	Non-causal	w: 1	majority	Directed alternating Secondary	Tests majority read concern against C3
C6	Non-causal	majority	local	Directed alternating Secondary	Tests majority write concern against C3
C8	Non-causal	majority	majority	Directed alternating Secondary	Strongest matrix concerns; still non-causal

The configurations form two connected experimental groups. C1-C4 are representative main configurations spanning session, routing, and concern choices. C3, C5, C6, and C8 form a controlled 2x2 matrix: all disable causal consistency and alternate reads across reachable Secondaries in the same deterministic order, while only write concern (w:1/majority) and read concern (local/majority) vary. C3 belongs to both groups, so its existing formal data is reused rather than rerun. The matrix attributes workload-visible differences to concern settings without treating majority as a general session guarantee.

5. Experimental Design

5.1 Workloads and Measurement

Each test document carries a monotonically increasing version, writer/run identifiers, and dependency fields. The runner records operation start/end times, results, errors, serving member, and model verdicts, then emits raw JSONL, summary JSON, and Markdown. Every formal baseline/stable-fault run executes 500 sequences per configuration-model pair for seeds 20260830, 20260831, and 20260832, yielding 1,500 principal checks. Database operation errors are counted separately from consistency violations: an error is an unavailable request, while a violation is evaluated only on a successful, decidable history.

Scenario	Fault or control	Purpose
S0	No fault	Normal-operation baseline
S1	Stop one Secondary	Stable behavior with a live majority
S2	Stop old Primary; wait for election	Post-election two-member topology
S3	Disconnect old Primary from majority and client	Stable state after a primary-side partition
T1	Continuous RYW during Primary stop	Consistency and availability during election
T2	Continuous RYW during Primary partition	Partition-triggered election window
S4	Freeze apply on one readable Secondary	Amplify explainable stale reads

5.2 Controlled Comparisons and Reuse

The representative group asks how realistic configuration bundles behave; the controlled matrix asks what changes when only read/write concern changes. Its corners are C3 (w:1/local), C5 (w:1/majority), C6 (majority/local), and C8 (majority/majority). Every matrix configuration uses a non-causal session and the same directed alternating-Secondary route, making concern comparisons valid. C2 remains a useful strong causal reference, but its Secondary is selected by the driver, so it is not presented as a strict one-variable session ablation against C8. The follow-up remote suite runs only C5, C6, and C8; C3 is merged at analysis time.

5.3 Fault Injection and Recovery

S1 and S2 stop the container holding the selected role. S3 uses docker network disconnect to isolate the pre-fault Primary from the replica-set network; the remaining two members retain a majority and elect a new Primary. T1/T2 start concurrent loops for the selected configurations before fault injection and label requests by operation start time as pre, election, or post. S4 enables enableTestCommands=1 only in a test-specific Compose override and runs rsSyncApplyStop on one Secondary. The non-causal matrix still follows its fixed alternation across two Secondaries, so each configuration encounters the paused and current members under the same schedule; this is controlled replication lag, not a claim that every read is pinned to one node. Apply processing is then resumed, catch-up verified, and normal containers recreated. Every scenario ends with a healthy-three-member check.

Rationale: S0-S3 isolate stable-state consistency, T1/T2 expose the transient unavailability hidden by waiting for election completion, and S4 provides a controlled causal mechanism showing that C3 anomalies arise from replica lag rather than test noise.

6. Results

6.1 S0-S3: Stable-State Matrix

S0 completed 60,012 database operations and 18,012 consistency checks with 0 errors. S1, S2, and S3 completed 18,012, 18,012, and 18,012 checks respectively, with 0, 0, and 0 database-operation errors. C1, C2, and C4 recorded 0 violations combined across every model and scenario; the only stable C3 anomaly was RYW.

Scenario	C3 RYW	Rate	Other C3 models	C1/C2/C4
S0	1,141 / 1,500	76.07%	MR 109/1,500 (7.27%); MW 0; WFR 0	0
S1	1,035 / 1,500	69.00%	MR 0; MW 0; WFR 0	0
S2	969 / 1,500	64.60%	MR 0; MW 0; WFR 0	0
S3	1,049 / 1,500	69.93%	MR 0; MW 0; WFR 0	0

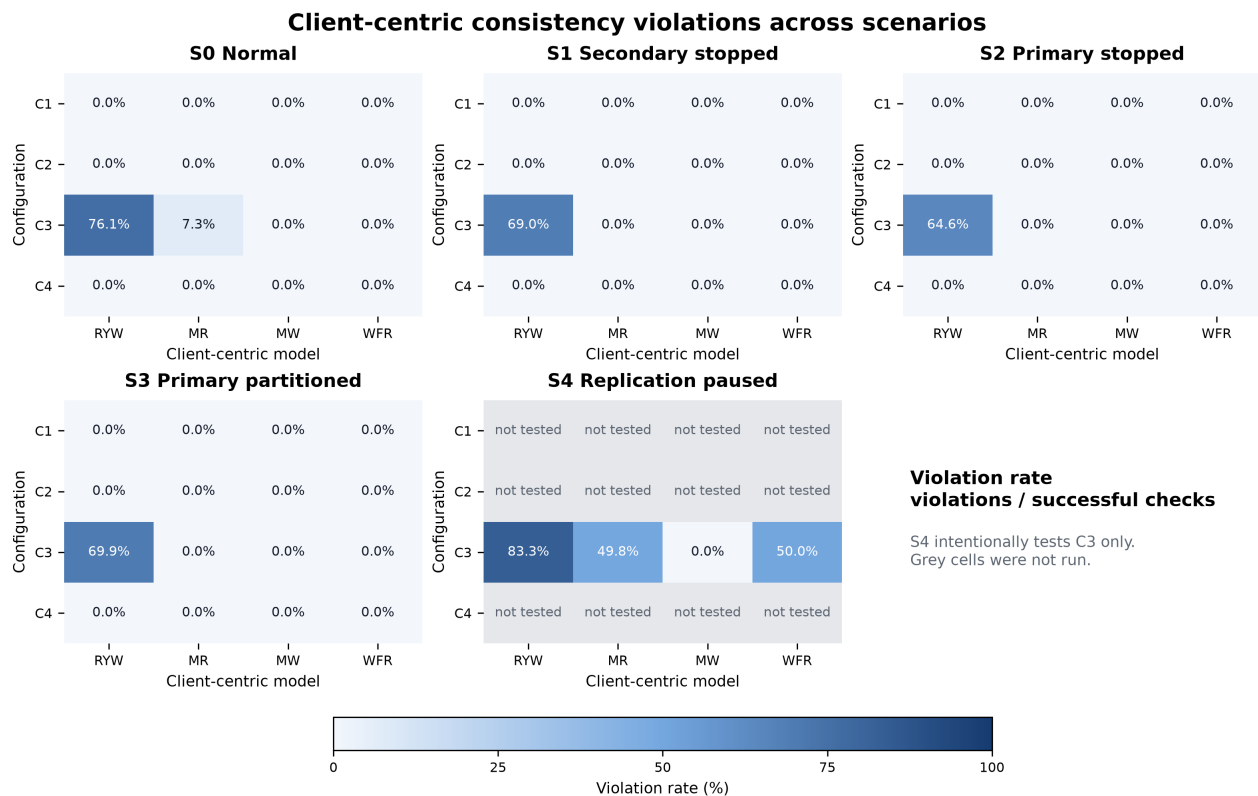


Figure 2. Raw checker rates for S0-S4; the S4 WFR 50% is an invalid proxy, not an established violation rate.

The slightly lower C3 RYW rates in S1-S3 do not mean faults strengthened C3. The number of available replicas, the selected Secondary, and runtime scheduling changed the chance of choosing a stale member. These rates are empirical frequencies in this deployment, not protocol guarantees.

6.2 Non-Causal Read/Write Concern Matrix

This comparison is not a separate experiment with a new logic bolted onto the main study. C3 is the bridge that completes the matrix with three missing concern combinations. All four configurations are non-causal and use the same directed alternating-Secondary route, so the table compares the empirical effect of read and write concern directly across scenarios.

Config	Read/write concern	S0 observation	S1-S3 observation	S4 observation
C3	local / w:1	RYW 76.07%; MR 7.27%	S1: RYW 69.00%; S2: RYW 64.60%; S3: RYW 69.93%	RYW 83.33%; MR 49.80%; WFR* 50.00%
C5	majority / w:1	RYW 97.87%; MR 5.93%	S1: RYW 100.00%; S2: RYW 99.93%; S3: RYW 100.00%	RYW 99.93%; MR 49.80%; WFR* 50.00%
C6	local / majority	RYW 20.73%; MR 7.40%	S1: RYW 0.33%; S2: RYW 2.87%; S3: RYW 1.27%	RYW 51.20%; MR 49.80%; WFR* 50.00%
C8	majority / majority	RYW 41.47%; MR 7.87%	S1: RYW 12.20%; S2: RYW 9.60%; S3: RYW 12.60%	RYW 58.87%; MR 49.80%; WFR* 50.00%

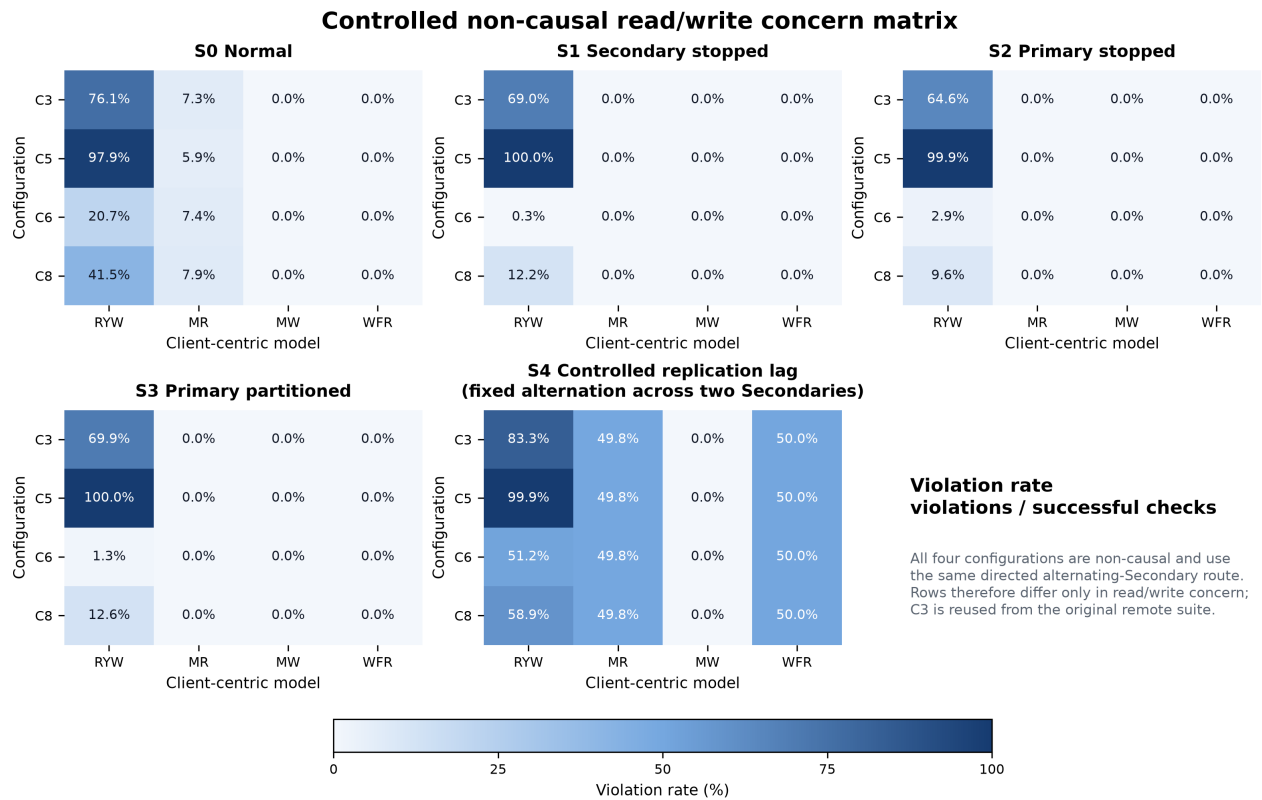


Figure 3. Raw checker rates for the non-causal 2x2 matrix; S4 WFR cells are invalid proxy flags, not established violations.

* In S4, the paused Secondary can return no document because replication stopped before document creation. The checker encodes this as parent_version = -1 and counts it directly. The near-50% WFR value is retained only as raw output, not used for a WFR consistency conclusion.

Under S4, RYW violation rates were C3 83.33%, C5 99.93%, C6 51.20%, and C8 58.87%. C3→C5 and C6→C8 isolate raising read concern from local to majority; C3→C6 and C5→C8 isolate raising write concern from w:1 to majority. Whatever the measured magnitude, the interpretation boundary is the same: majority write concern addresses acknowledged durability and majority read concern restricts visibility, but without a causal session neither alone nor together is a general RYW/MR/WFR guarantee when reads can reach a lagging Secondary.

6.3 Latency

Config	S0 p50 / p95 (ms)	S1 p50 / p95	S2 p50 / p95	S3 p50 / p95
C1	3.19 / 9.24	2.09 / 5.84	2.23 / 7.09	2.05 / 6.07
C2	3.43 / 9.11	2.21 / 6.62	2.84 / 9.73	2.15 / 7.04
C3	1.48 / 5.00	1.01 / 3.35	1.07 / 3.14	0.88 / 2.85
C4	3.22 / 9.13	2.25 / 6.79	2.24 / 6.65	2.17 / 6.45

The table reports end-to-end latency for one RYW write-read pair. C3 is fastest, consistent with w:1/local avoiding majority waits, but this accompanies weaker visibility. C2's baseline WFR p95 was 42.27 ms versus 19.37 ms for C1, consistent with a causal secondary read sometimes waiting to satisfy afterClusterTime. Lower medians in some fault scenarios reflect container, cache, and runtime variation and should not be interpreted as fault-induced optimization.

6.4 T1/T2: Election Transition Windows

Scenario	Checks	Violations	Op. errors	Mean election	Strong successful ops
T1	9,635	2,420 (all C3)	34 (all election)	11,334 ms	0 violations
T2	10,974	2,532 (all C3)	60 (all election)	15,041 ms	0 violations

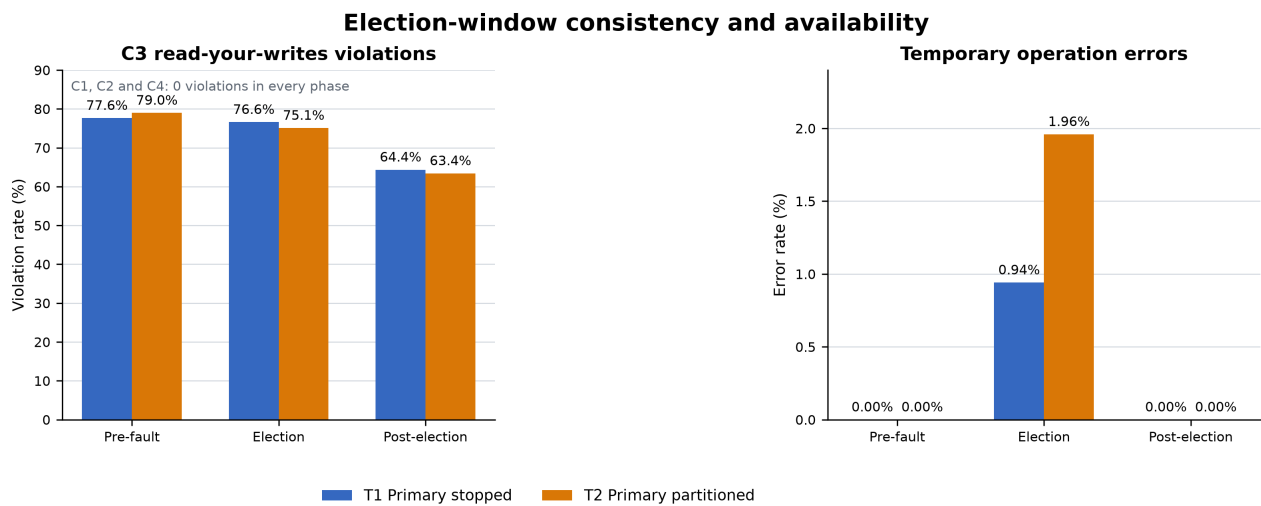


Figure 4. Successful checks, violations, and operation errors by configuration in T1/T2.

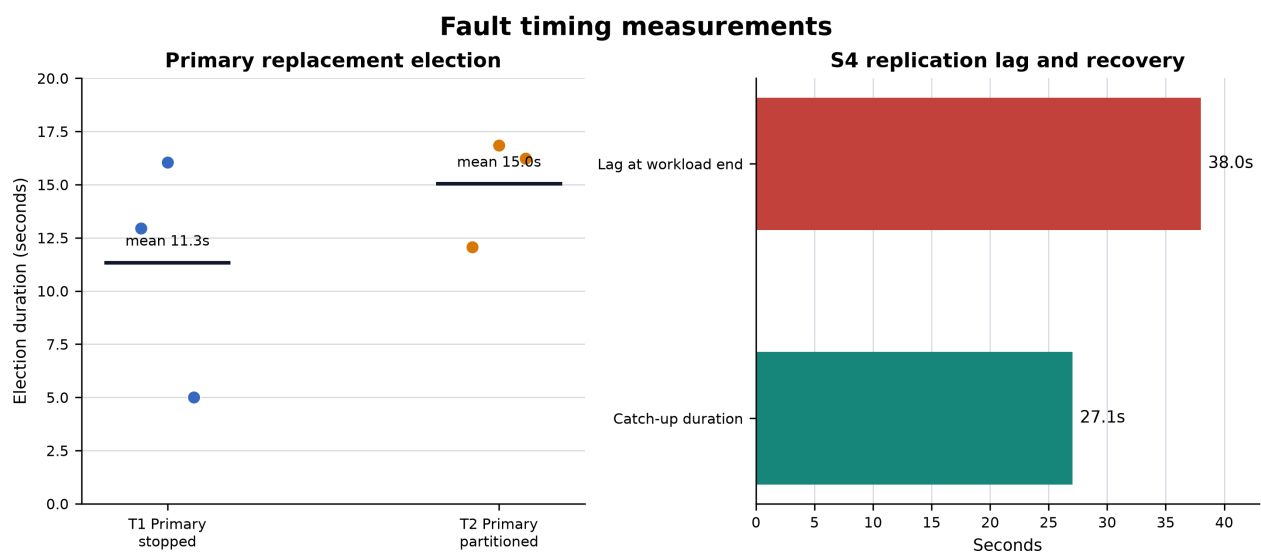


Figure 5. Election timing for S2, S3, T1, and T2. T1/T2 show all three seeds.

Scenario	pre	election	post
T1 C3	771/993 = 77.64%	537/701 = 76.60%	1,112/1,728 = 64.35%
T2 C3	849/1,075 = 78.98%	413/550 = 75.09%	1,270/2,003 = 63.40%

The transition tests give two complementary conclusions. First, successful RYW pairs under C1, C2, and C4 recorded 0 violations combined. Second, 34 and 60 operation errors occurred during elections, showing a temporary availability cost when no writable Primary could be selected immediately. C3 was already weak before each fault, with similar pre and election violation rates; the election did not create a new C3 consistency defect, but mainly added transient failures. The per-seed election-window error count varied noticeably (T1: 3-27; T2: 16-24), which on a single shared VM more likely reflects scheduling noise than a real difference in protocol behaviour, so it is reported here rather than treated as a stable quantitative result.

For the three new matrix corners in the same remote environment, transition-window RYW violations/successful checks were: C5: T1 3,958/4,141, T2 3,486/3,646; C6: T1 159/2,992, T2 164/2,517; C8: T1 543/2,993, T2 589/2,522. C3's original transition data is already included above and was not rerun. These results test whether concern combinations change stale successful histories during elections; operation errors remain separate from consistency violations.

6.5 S4: Controlled Replication Lag

Model	Violations / checks	Rate	Interpretation
RYW	1,250 / 1,500	83.33%	Post-write read hits the frozen Secondary.
MR	747 / 1,500	49.80%	Alternation between current and frozen secondaries.
MW	0 / 3 validations	0.00%	Primary accepts ordered writes; final state remains ordered.
WFR*	750 / 1,500	50.00%	Null read encoded as -1; raw flags, not valid violations.

C3 under a readable but replication-paused Secondary

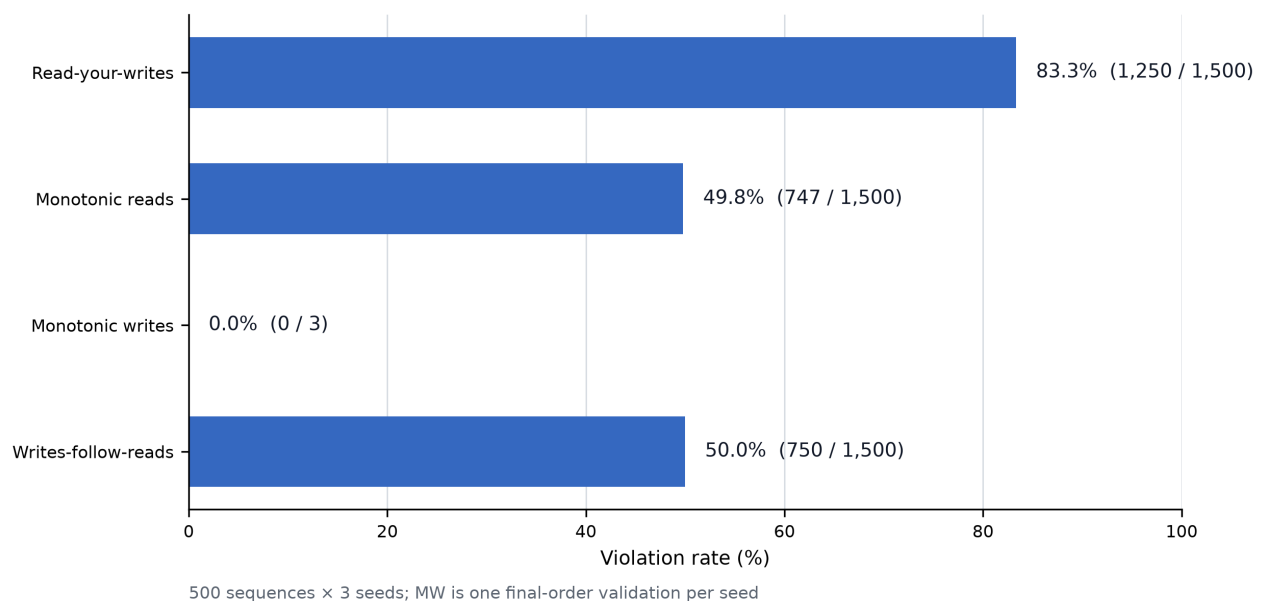


Figure 6. C3 raw S4 checker rates; WFR 50% is an invalid proxy flag, not a violation rate.

The S4 raw summary contains 4,503 checks, 0 database-operation errors, and 2,747 checker flags (61.00%). Of these, 750 C3 WFR flags are not valid consistency violations, so the combined count and rate are not formal violation statistics. The frozen member was 38 seconds behind at test end; after apply resumed, three consecutive checks found lag within one second and full recovery took 27,051 ms. MR near 50% fits alternating Secondary reads; WFR near 50% mainly reflects the checker assigning -1 to a null read.

7. Predictions Versus Observations

7.1 Representative Configurations

Config	Prediction	Observation	Assessment
C1	All four hold	Zero violations in S0-S3 and successful T1/T2 RYW	Agrees
C2	Cross-member causal session holds	Zero violations on Secondary reads; some higher tail latency	Agrees
C3	RYW/MR/WFR may fail; MW usually holds	S0 exposed RYW/MR; S4 exposed RYW/MR; WFR flags invalid	RYW/MR agree; WFR inconclusive
C4	Holds here, not a general causal guarantee	Zero violations in this workload	Supported within scope

The key distinction is safety versus availability. The strong configurations did not convert election-time failures into consistency anomalies: unsafe operations failed or retried, while successful histories continued to satisfy RYW. C3 instead offered low-latency, highly available reads that could successfully return stale versions.

7.2 Controlled Concern Matrix

Config	Isolated comparison	Theoretical boundary	Observation (S0 / S4)
C3	Matrix baseline: local + w:1	No general session guarantees; workload likely preserves MW	RYW 76.07%; MR 7.27% / RYW 83.33%; MR 49.80%; WFR* 50.00%
C5	Only read concern raised from C3	Majority read does not mean the client's latest write	RYW 97.87%; MR 5.93% / RYW 99.93%; MR 49.80%; WFR* 50.00%
C6	Only write concern raised from C3	Majority write protects durability, not Secondary apply recency	RYW 20.73%; MR 7.40% / RYW 51.20%; MR 49.80%; WFR* 50.00%
C8	Majority/majority intersection of C5/C6	Concern combination still does not replace a causal session	RYW 41.47%; MR 7.87% / RYW 58.87%; MR 49.80%; WFR* 50.00%

* S4 WFR values are raw checker flags, not valid violation rates; the prediction-observation comparison relies on RYW, MR, and MW.

The matrix is valuable even if its four rows do not produce four radically different outcomes. By holding session and routing constant, it tests whether concern alone changes the histories visible to the client. If majority/majority still returns a successful stale read, that is the result: durability and committed visibility do not automatically provide cross-replica session ordering.

8. Limitations and Threats to Validity

- All members share one VM, hardware, disk, and underlying network; container isolation is not equivalent to three physical or virtual hosts.
- S1-S3 run the full matrix after election completion. T1/T2 cover only RYW continuously during the transition; the other three models were not stressed through elections.
- S3 isolates the old Primary from both peers and the client. It does not create split-client histories with clients on both partition sides.
- S4 uses enableTestCommands and rsSyncApplyStop as an extreme diagnostic failpoint, not as a realistic latency distribution.
- Only three seeds are used. Workload shape, driver selection, and local scheduling influence empirical rates. Zero observed violations is not a mathematical proof.
- Phases are assigned by operation start time; an operation crossing an election boundary still belongs to one phase.
- MW performs 500 sequential writes per configuration and seed, but the summary denominator counts one final history validation per seed; therefore the table reports three validations.
- C3 comes from the earlier main-suite remote run, while C5/C6/C8 come from a later run on the same GCP VM. Software and deployment settings match, but cache and host scheduling can still affect absolute latency across collection times; violation patterns are therefore more reliable than small latency differences.
- S4 pauses one Secondary while matrix configurations alternate reads across two Secondaries in a fixed order. Every configuration receives the same routing schedule, but this is not a strict operation-by-operation pairing in which every read targets the paused member.
- The S4 WFR checker has a measurement-validity defect. The paused Secondary stopped applying oplog entries before the test document was created, so a read there can return no document. The code maps that null read to parent_version = -1 and flags a violation solely because the value is negative. A null read provides no observed version to order a later write against and does not show that the write executed on an older state. Thus the approximately 50% WFR figure is an invalid proxy, not a formal violation rate. With writes routed to a single Primary that is normally no older than a version read from a Secondary, true WFR violations are not expected in this workload, but this experiment did not validly test that expectation.

9. Reproducibility

9.1 Main Commands

```
# Baseline S0
docker compose run --rm --no-deps runner python -m experiments.run_baseline --iterations 500 --seeds
20260830,20260831,20260832 --label formal

# Stable fault scenarios (replace the scenario name for S2/S3)
./scripts/run_fault_scenario.sh S1-secondary-failure 500 20260830,20260831,20260832 formal

# Transition windows; repeat for seeds 20260830-20260832
./scripts/run_transition_scenario.sh T1-primary-stop-transition 20260830 formal 35 3
./scripts/run_transition_scenario.sh T2-primary-partition-transition 20260830 formal 35 3

# Controlled replication lag for the original representative suite
./scripts/run_replication_lag_scenario.sh 500 20260830,20260831,20260832 formal

# Complete the non-causal concern matrix on the same remote VM.
# This resumable suite runs only C5/C6/C8 and reuses the existing C3 data.
./scripts/run_noncausal_concern_matrix.sh noncausal-remote-v2 500 20260830,20260831,20260832

# Recovery and report figures
./scripts/restore_cluster.sh
./scripts/cluster_status.sh
python3 -m analysis.generate_report_figures
```

The repository contains Compose configuration, initialization and fault scripts, Python workloads, formal summary JSON, result narratives, and chart-generation code. Raw JSONL can be regenerated with the commands above. To avoid unintended compute charges, stop the VM in Google Cloud after experiments and verify TERMINATED status; persistent disk may continue to incur storage charges.

9.2 Formal Run Identifiers

Scenario	Selected formal run ID / suffix
S0	baseline-s0-formal-20260830T154711Z-3c931c32
S1	s1-secondary-failure-formal-20260830T181846Z-8e657452
S2	s2-primary-failure-formal-20260830T182320Z-f604f5eb
S3	s3-primary-partition-formal-rerun-20260830T183455Z-59f3d378
S4	s4-secondary-replication-lag-formal-20260831T081523Z-85b1e3b3
T1	6cad906c; 867b572c; ae94f015
T2	0609280f; b9f2cd1b; ceb526e5
S0 (C5/C6/C8)	s0-normal-noncausal-remote-v2-20260916T040639Z-8847752f
S1 (C5/C6/C8)	s1-secondary-failure-noncausal-remote-v2-20260916T041007Z-f2701836
S2 (C5/C6/C8)	s2-primary-failure-noncausal-remote-v2-20260916T041333Z-db219dd1
S3 (C5/C6/C8)	s3-primary-partition-noncausal-remote-v2-20260916T041729Z-8aded5fc
S4 (C5/C6/C8)	s4-secondary-replication-lag-noncausal-remote-v2-20260916T043039Z-e309d4fc
T1 (C5/C6/C8)	49d8cfb0; 9e0d2c73; 75c23be8
T2 (C5/C6/C8)	0c2b4e12; 78d03479; 0952edec

10. Conclusion

The project completes a full cycle from theoretical prediction through replica-set deployment, tunable configurations, four executable model workloads, fault injection, and mechanistic validation. The representative configurations show the end-to-end interaction of causal sessions, routing, and concern settings. The non-causal C3/C5/C6/C8 matrix then holds session and Secondary order fixed, allowing read and write concern to be compared without rerunning C3. The results separate durability, committed visibility, session ordering, and availability: majority settings do not automatically equal cross-replica client-centric guarantees, elections primarily caused temporary unavailability, and S4 amplified successful stale reads into mechanistic evidence.

References

- [1] MongoDB Manual, Read Isolation, Consistency, and Recency. <https://www.mongodb.com/docs/manual/core/read-isolation-consistency-recency/>
- [2] MongoDB Manual, Read Concern. <https://www.mongodb.com/docs/manual/reference/read-concern/>
- [3] MongoDB Manual, Write Concern. <https://www.mongodb.com/docs/manual/reference/write-concern/>
- [4] MongoDB Manual, Read Preference. <https://www.mongodb.com/docs/manual/core/read-preference/>
- [5] MongoDB Manual, Self-Managed Replica Set Configuration. <https://www.mongodb.com/docs/manual/reference/replica-configuration/>
- [6] MongoDB PyMongo Driver Documentation, Transactions and Causal Consistency. <https://www.mongodb.com/docs/languages/python/pymongo-driver/current/crud/transactions/#causal-consistency>
- [7] Docker Documentation, Docker Compose. <https://docs.docker.com/compose/>
- [8] Docker CLI Documentation, docker network disconnect. <https://docs.docker.com/reference/cli/docker/network/disconnect/>
- [9] Google Cloud Documentation, Compute Engine Instances. <https://cloud.google.com/compute/docs/instances>
- [10] DSA5208 Lecture 3: Replication and Consistency (course material supplied by the instructor).

All online documentation was last accessed on 31 August 2026.

AI Usage Statement

OpenAI ChatGPT/Codex supported concept explanation, experiment planning, script and report editing, result checks, and figure generation. Anthropic Claude (Claude Code) earlier reviewed reproducibility and helped implement a causal-session C5-C8 extension; this historical output was excluded because its controls and environment differed from the final design. Codex then helped build the non-causal C3/C5/C6/C8 matrix and run the missing C5/C6/C8 combinations on the main GCP VM. Project members supervised cloud operations, fault injection, validation, and interpretation. All reported measurements are stored experimental outputs, not AI-generated.